INSTITUTO FEDERAL
Paraná
Campus Pinhais

INSTITUTO FEDERAL DO PARANÁ
CAMPUS PINHAIS
CURSO DE BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

GUSTAVO KÊM FUKUDA
RAFAEL NOGUEIRA

RELATÓRIO DO TRABALHO DE ESTRUTURA DE DADOS II

PINHAIS
2026

Tema 3: Sistema de inventário

Este relatório tem como objetivo mostrar um sistema de inventário em java implementado com funções Hash(para distribuir e buscar jogadores), Heaps(para organizar os itens por raridade) e gerador de números aleatórios(para a criação de itens).

1.Tamanho

O tamanho escolhido para a tabela hash é dada por uma função que pega o número total de jogadores multiplica por 2 e procura o número primo superior mais próximo. Essa função é usada para evitar que ocorram muitas colisões.

2.Função Hash

Para a função Hash foi usada o hashCode nativo do java combinado com o método da divisão

A função hashCode utiliza um multiplicador primo(31), que ajuda a espalhar os valores de forma que nicks parecidos gerem valores inteiros diferentes. Ele é o gerador do valor base.

O método da divisão consiste em pegar o valor numérico e usar o módulo “%” (resto da divisão) pelo tamanho da array de jogadores. Dessa forma o cálculo evita dar um endereço que ultrapasse o tamanho da array.

3.Funções de espalhamento

Os métodos escolhidos foram a Linear Probing(Sondagem Linear) e Quadratic Probing(Sondagem Quadrática)

Linear Probing:A Sondagem Linear foi escolhida pela sua simplicidade e pela excelente localidade de referência, aproveitando o cache do processador ao buscar elementos em posições de memória contíguas. Sua execução se dá por um índice que vai sendo somado a cada iteração do jogador, incrementando o “ValorNumerico” para empurrar o Jogador 1 casa para frente.
Quadratic Probing: A Sondagem Quadrática foi implementada para solucionar o

problema de Agrupamento Primário (Primary Clustering) inerente à sondagem linear. Ao utilizar um deslocamento baseado em i^2, o algoritmo consegue ‘saltar’ regiões densamente povoadas da tabela, espalhando os jogadores de forma mais eficiente.

4.Heaps

O método Heap utilizado foi o MaxHeap. A justificativa para a escolha de tal método se deve ao fato de que o requisito necessário para o exercício era um heap que organizasse os itens por raridade(do maior para o menor), o qual os outros métodos de Heap não se encaixavam no cenário.

5.Números Aleatórios

O método utilizado para gerar números aleatórios foi o LGC

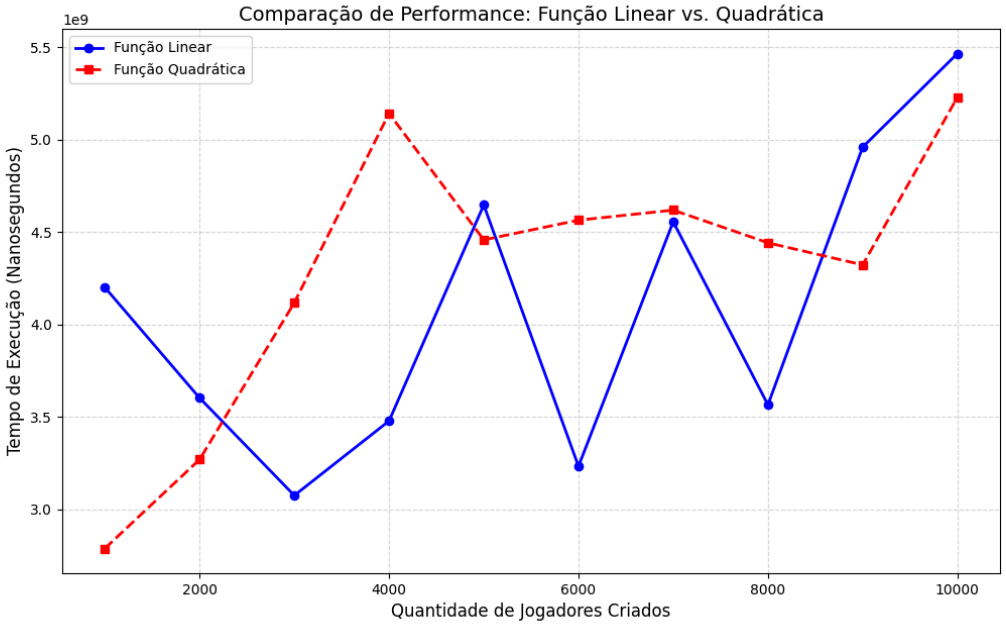
6.Comparações

Análise da Comparação

Alta instabilidade e ruído nos dados. Em um cenário teórico ideal, se espera que a linha azul fosse uma reta ascendente suave e a linha vermelha uma curva parabólica evidente. No entanto, ambas as funções apresentam oscilações significativas

A Função Linear exibe picos e vales acentuados, chegando a ser mais lenta que a quadrática em 5.000 e 9.000 jogadores, mas apresentando quedas bruscas de tempo em outros pontos..

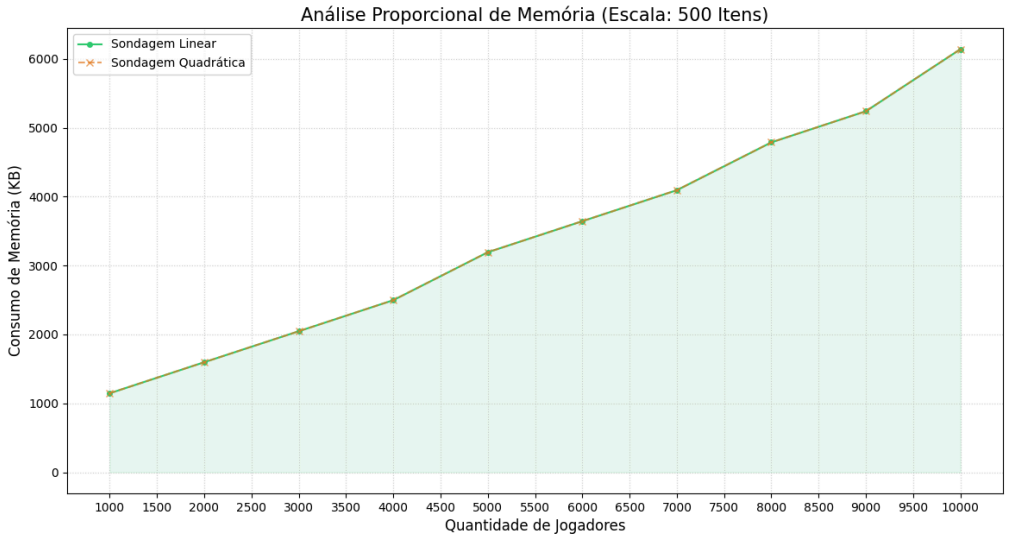
A Função Quadrática começa com um desempenho superior (mais rápido), mas sofre um salto drástico de tempo entre 2.000 e 4.000 jogadores, mantendo-se relativamente instável a partir daí.



Conclusões:

Conclui-se que houve certo ruido no teste, principalmente pelo fator da gestão de memória realizada pela linguagem Java.

Uso de memória ao inserir jogadores

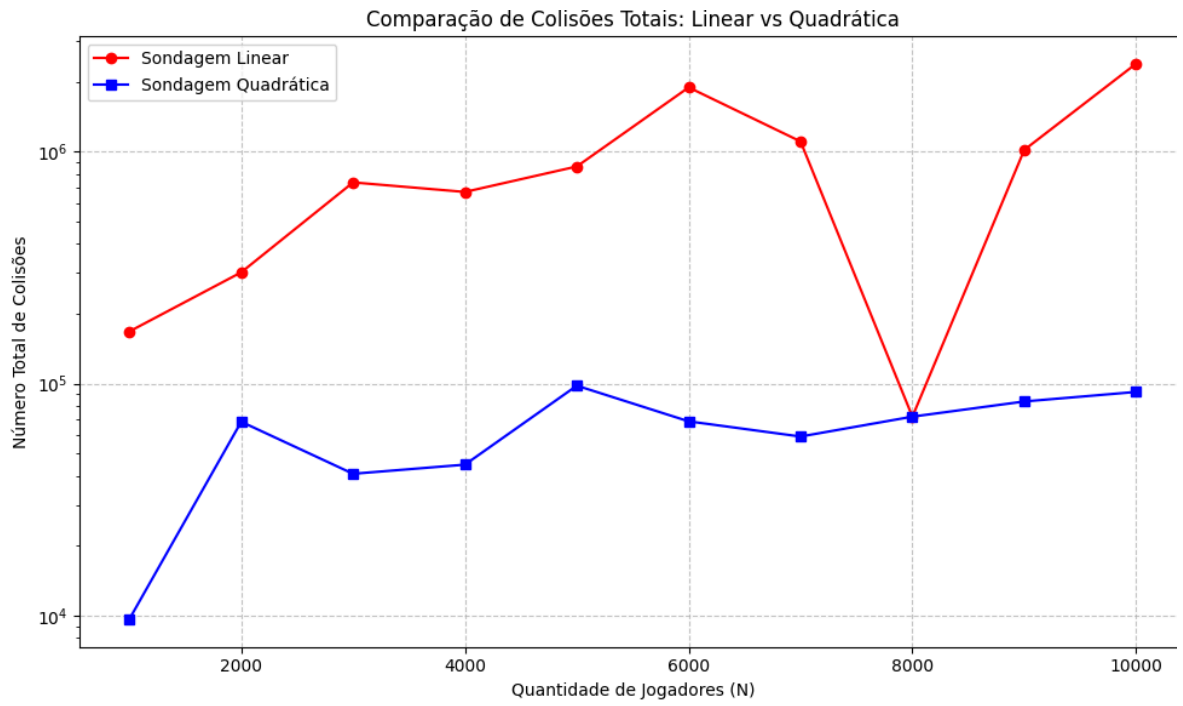


Por conta da linguagem Java o uso de memória de utilizada pelos 2 métodos é quase idêntica, pois o modo que o Java aloca o uso de memória é feita de um jeito diferente comparada às outras linguagens.

Colisões de jogadores

Sondagem Quadrática (Azul): Mantém-se consistentemente abaixo da linha vermelha na maior parte do tempo. Isso indica que, para este conjunto de dados e testes, ela foi mais eficiente (levou menos tempo ou realizou menos operações). Sua curva é mais estável, embora ainda apresente pequenas oscilações.

Sondagem Linear (Vermelho): Apresenta um tempo de execução significativamente maior e uma volatilidade muito alta. Note o pico de queda brusca próximo ao final do gráfico, seguido de uma subida imediata. Esse comportamento errático é típico de situações onde ocorrem muitos agrupamentos de chaves (*clustering*).

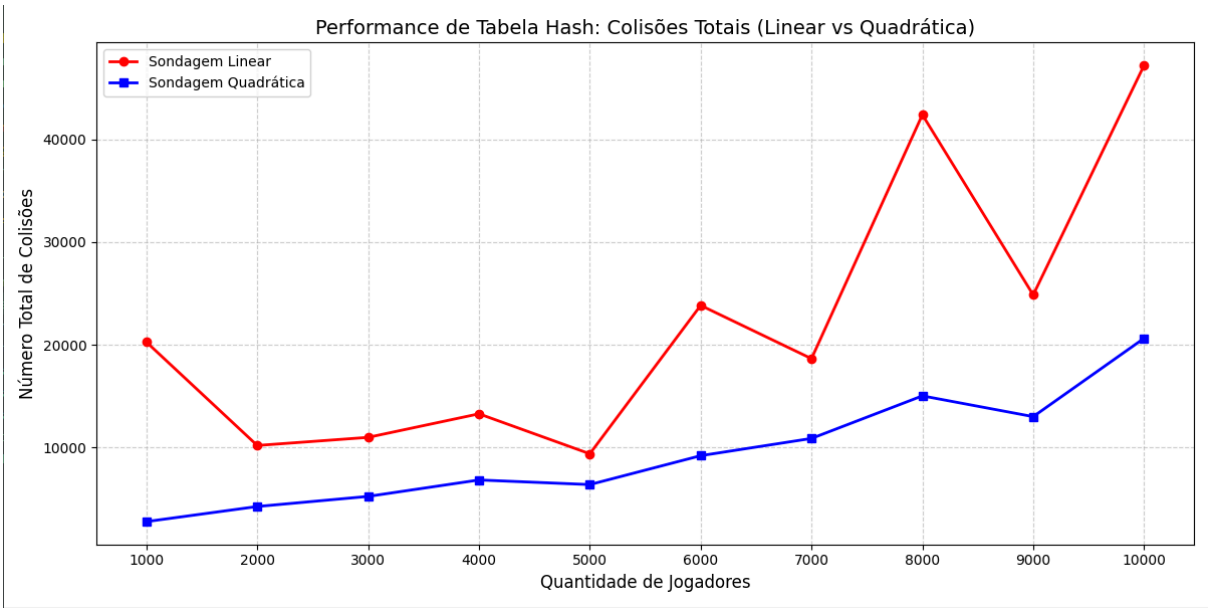


Conclusões do Gráfico

Superioridade da Sondagem Quadrática: O gráfico sugere que a sondagem quadrática é a melhor escolha para evitar o problema de agrupamento primário. Como ela pula os índices de forma não linear, ela consegue encontrar espaços vazios na tabela de forma mais distribuída do que a linear.

Problema de Agrupamento (Clustering): A linha vermelha elevada e instável é um forte indicativo de que a Sondagem Linear sofreu com colisões em sequência. Quando várias chaves são mapeadas para índices próximos, a busca por um espaço livre se torna cada vez mais lenta, gerando esses "degraus" de tempo.

Anomalia de Teste: O ponto onde a linha vermelha cai drasticamente até quase tocar a azul sugere um caso isolado onde a busca foi extremamente rápida (talvez uma inserção em uma tabela quase vazia ou uma chave que caiu exatamente em um espaço livre de primeira), mas a tendência geral continua sendo de baixa performance.



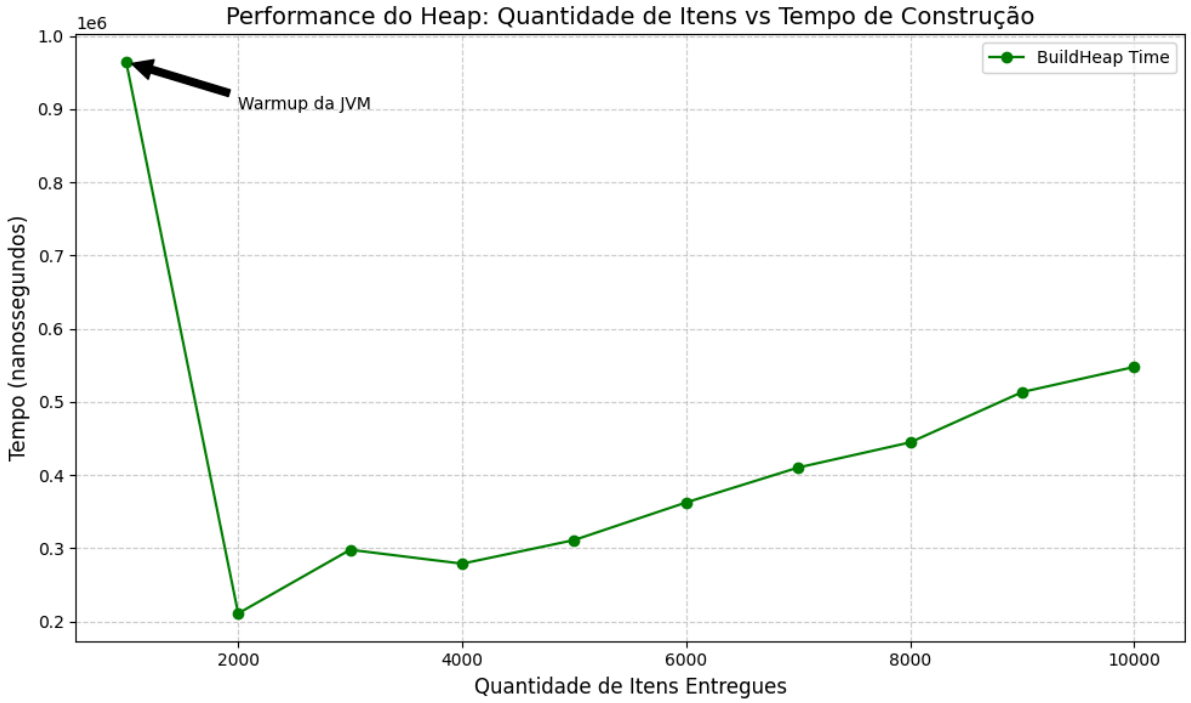
Ao compararmos os dois métodos, observamos que a Sondagem Quadrática apresentou desempenho superior em todos os cenários testados.

No cenário de alta ocupação (sem redimensionamento), a Sondagem Linear sofreu uma degradação severa devido ao fenômeno de Agrupamento Primário, enquanto a Quadrática manteve tempos de inserção mais baixos ao distribuir as chaves de forma não contígua.

Surpreendentemente, mesmo no cenário de baixa ocupação (Tamanho * 2), a Sondagem Quadrática superou a Linear. Isso demonstra que, embora a Linear tenha uma melhor localidade de cache, a eficácia da Quadrática em evitar colisões sucessivas compensa o custo computacional do cálculo quadrático, tornando-a a escolha mais eficiente para o nosso sistema de inventário.

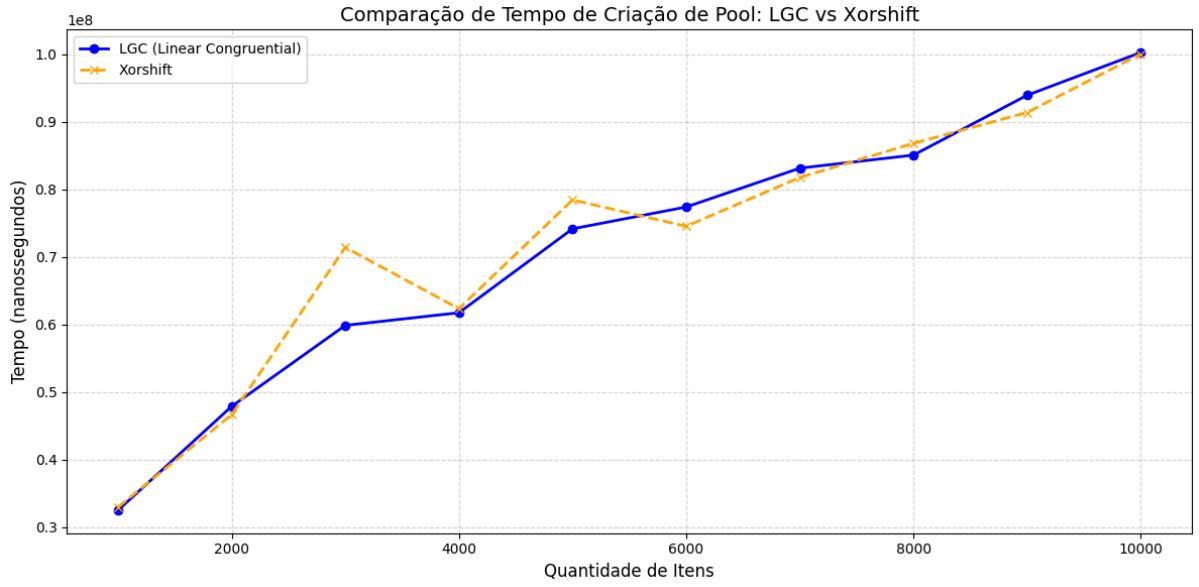
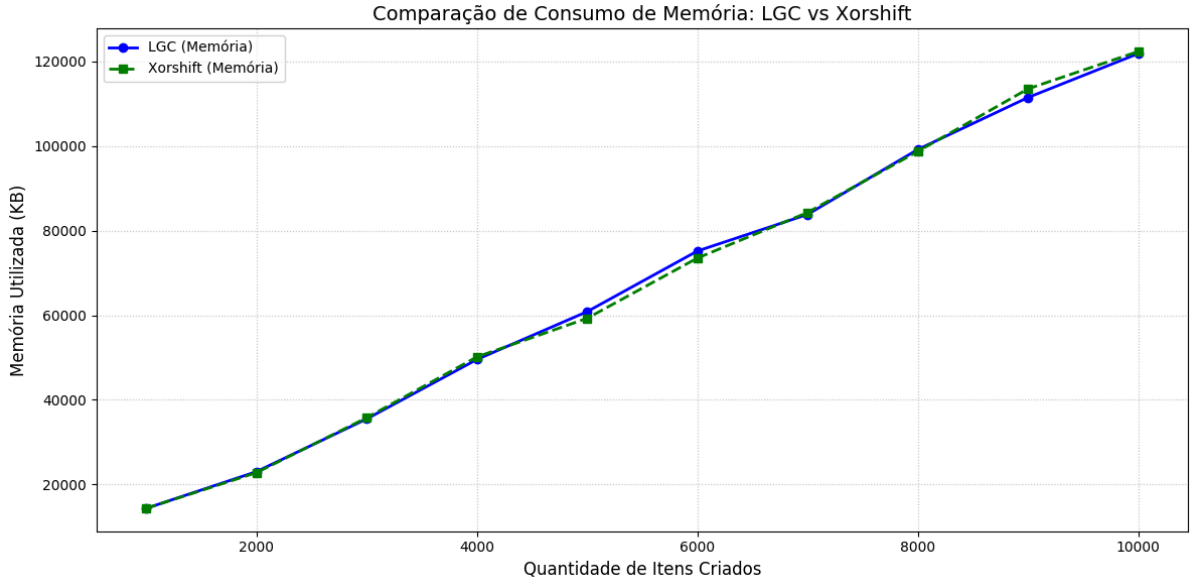
Nota: em teoria no sem tamanho * 2 A sondagem Linear deveria ganhar, pois se as array estão quase todas ocupadas, a sondagem conseguiria avançar com mais facilidade nos buracos que sobrariam, já a quadrática demoraria mais iterações para conseguir pular nesse buraco já que ela não segue uma sequência linear. Provavelmente esses erros acontecem por conta do hashCode do Java que pode gerar valores perto um do outro por coincidência.

Heap



Conclusão: Por conta da linguagem Java, O JVM demora mais para realizar a primeira execução, e após isso ela fica otimizada, por isso a queda brusca e a constância depois

Comparação de tempo dos números aleatórios LGC vs Xorshift



Comparação de consumo de memória LGC vs Xorshift